

Engineering Excellence & Software Reliability Policy

Document ID: POL-ENG-2026-004

Effective Date: June 1, 2026

Version: 2.1

Owner: VP of Engineering

1. Objective & Scope

This directive defines the fundamental software quality assurance, continuous integration, and engineering standards mandatory across all code repositories owned, operated, or maintained by B-Cubed. Maintaining highly reliable, deterministic, and clean codebases directly drives our architectural resilience and protects collaborative efficiency.

This policy applies universally to all product squads, full-time engineering staff, technical mentors, and third-party contract developers contributing to our software ecosystems.

2. Static Analysis & Linting Framework

To guarantee code readability and prevent syntax anomalies before runtime execution, all engineering environments must enforce active static analysis. Static analysis operates as an automated gatekeeper safeguarding production stability.

2.1 Core Linting Standards

- **Primary Engine:** ESLint is designated as the unified JavaScript and TypeScript static analysis tool. Alternative rulesets must be approved explicitly by the Architecture Guild.
- **Instant Verification:** Local IDE environments must integrate real-time ESLint formatting triggers (such as on-save hooks) to catch type assignment errors and unhandled reference faults instantly during development.

Critical Verification Example

Consider the logic failure where an immutable assignment is broken:

```
const userRole = "Mentor";  
userRole = "Lead"; // ESLint Instantly Triggers: TypeError: Assignment to constant  
variable.
```

Static analysis mitigates this issue inside the IDE before a unit test ever runs or a compilation step begins.

2.2 The 5-Step Static Analysis Process

Every engineering team must apply the standardized lifecycle for linting configuration:

1. **Define Needs:** Establish project-specific strictness levels, syntax requirements, and target formatting frameworks.
2. **Choose Tools:** Install the standard B-Cubed dependencies, primarily `eslint` and contextual configurations via NPM.
3. **Configure:** Implement an unmodifiable, project-rooted `.eslint.config.js` setting exact formatting rules.
4. **Integrate:** Embed execution routines inside local developer commands and native text-editor extensions.
5. **Monitor:** Ensure regular auditing during production pull-requests through continuous pipeline gates.

3. Automated Unit Testing Directives

A high-performing software platform requires predictable modular behaviors. All core algorithmic subroutines, helper utilities, and processing blocks are mandated to possess atomic coverage profiles.

3.1 Testing Ecosystem

Jest is standard for front-end modules and decoupled backend services. Its zero-configuration, robust matcher interface, and isolated test-runner model optimize velocity.

3.2 Methodology Dichotomy: TDD vs. BDD

B-Cubed recognizes two balanced testing strategies based on product phase and implementation scope:

Attribute	Test-Driven Development (TDD)	Behavior-Driven Development (BDD)
Primary Focus	Functional, granular components & code structures.	User interactions, workflows, and business outcomes.
Lifecycle Sequence	Write Test → Observe Failure → Code Minimal Fix → Refactor.	Define Feature Narrative → Form Scenario Logic → Code System.
Target Audience	Core Software Developers and System Architects.	Product Owners, QA Teams, and Engineers.

3.3 Required Matchers & Structural Assertions

Engineers must assemble explicit testing logic inside standard `test()` or `it()` definitions, relying on strict mathematical assertions like `expect().toBe()` or deep object structures via `.toEqual()`.

4. Reliability Metrics & Deployment Barriers

No pipeline will generate an optimized build asset for release production if a singular Unit Test fails or if static analysis thresholds discover high-severity vulnerabilities. Reliability is defined as mathematical stability under continuous scaling pressure.